

Editor integration

Calepin documents are ordinary Typst source files, so they work with any editor that supports Typst. The editor can provide language tooling and preview, while Calepin remains responsible for executing computational chunks.

VS Code, Cursor, and Positron

Install *Calepin for Typst* from the VS Code Marketplace. Cursor, Positron, and other VSX-compatible editors can install it from Open VSX. The extension adds two command-palette actions for computational chunks:

- Typst: Calepin runs `calepin watch --eval-only` for the active document.
- Typst: Stop Calepin stops the watcher started by the extension.

The start command saves the active document first and uses the Python interpreter selected by the Python extension when available. The extension uses its bundled Calepin binary, then falls back to calepin on PATH; set `calepin.binaryPath` to select another executable. It has no editor-extension dependencies and does not provide or control preview, choose an output format, or forward Typst rendering arguments.

Tinymist preview

Tinymist previews the authored `.typ` file after one successful Calepin compile, with no notebook-specific `typstExtraArgs`:

```
calepin compile paper.typ
```

Use the canonical facade and document adapter:

```
#import "../.calepin/calepin.typ" as calepin
#show: calepin.document
```

Install Tinymist separately when you want its language tooling and preview. Start its preview independently of Calepin. Tinymist refreshes prose, layout, and other Typst changes, while the Typst: Calepin watcher evaluates computational chunks. The equivalent terminal command is:

```
calepin watch paper.typ --eval-only
```

This mode refreshes Calepin artifacts without launching a second `typst watch`. A lightweight metadata query checks relevant source changes, but prose-only edits do not rerun Python, R, Jupyter, shell, or diagram engines. Alternatively, run `calepin compile` manually after code changes. Until Calepin runs again, the preview shows the last stored computational snapshot.

The generic facade follows the most recently compiled or watched single notebook when Calepin is not supplying its internal Typst inputs. For independent simultaneous previews, import the generated notebook-specific facade instead:

```
#import "../.calepin/paper/calepin.typ" as calepin
#show: calepin.document
```

Existing aliased imports and Calepin-managed workflows remain compatible. Do not use `#import "../.calepin/calepin.typ": *`: the exported document adapter shadows Typst's built-in document element, including `#set document(...)`.

Other editors

The same compile-once contract works with other Typst language servers, preview tools, and plain `typst compile`, provided they use the same project root as Calepin. Run `calepin compile` to refresh stored results, or run `calepin watch paper.typ --eval-only` for automatic evaluation while the editor handles rendering. No editor-specific Calepin settings or lock file are required.